

Unit 3: Java Script

**JavaScript Basics – Functions – Arrays – DOM - Built-in Objects –
Regular Expression - Event handling – Validation**

Dr. Adyasha Sahu

JavaScript Basics

- JavaScript is the world's most popular programming language.
- JavaScript is the programming language of the Web.
- JavaScript is a versatile, high-level, interpreted programming language primarily known for enabling interactive and dynamic content on web pages. It is a cornerstone of modern web development, alongside HTML for structure and CSS for styling.

JavaScript Basics

- Why Study JavaScript?
- JavaScript is one of the **3 languages** all web developers must learn:
 1. [HTML](#) to define the content of web pages
 2. [CSS](#) to specify the layout of web pages
 3. **JavaScript** to program the behavior of web pages

JavaScript Basics

Key Features of JavaScript

- Here are some key features of JavaScript that make it a powerful language for web development:
- **Client-Side Scripting:** JavaScript runs on the user's [browser](#), so has a faster response time without needing to communicate with the [server](#).
- **Versatile:** JavaScript can be used for a wide range of tasks, from simple calculations to complex server-side applications.
- **Event-Driven:** JavaScript can respond to user actions (clicks, keystrokes) in real-time.
- **Asynchronous:** JavaScript can handle tasks like fetching data from [servers](#) without freezing the user interface.
- **Rich Ecosystem:** There are numerous libraries and frameworks built on JavaScript, such as [React](#), [Angular](#), and Vue.js, which make development faster and more efficient.

JavaScript Basics

- The <script> Tag
- In HTML, JavaScript code is inserted between <script> and </script> tags.

```
<!DOCTYPE html>
```

```
<html>
```

```
<body>
```

```
<h2>JavaScript in Body</h2>
```

```
<p id="demo"></p>
```

```
<script>
```

```
document.getElementById("demo").innerHTML = "My First JavaScript";
```

```
</script>
```

```
</body>
```

```
</html>
```

Output:

JavaScript in Body

My First JavaScript

JavaScript Basics

JavaScript in `<head>` or `<body>`

- You can place any number of scripts in an HTML document.
- Scripts can be placed in the `<body>`, or in the `<head>` section of an HTML page, or in both.

JavaScript Basics

JavaScript Functions and Events:

- A JavaScript function is a block of JavaScript code, that can be executed when "called" for.
- For example, a function can be called when an **event** occurs, like when the user clicks a button.
- A JavaScript **function** is a block of JavaScript code, that can be executed when "called" for. For example, a function can be called when an **event** occurs, like when the user clicks a button.

JavaScript Basics

JavaScript in <head>

- In this example, a JavaScript function is placed in the <head> section of an HTML page.
- The function is invoked (called) when a button is clicked:

JavaScript Basics

```
<!DOCTYPE html>
<html>
<head>
<script>
function myFunction() {
  document.getElementById("demo").innerHTML = "Paragraph changed.";
}
</script>
</head>
<body>

<h2>Demo JavaScript in Head</h2>

<p id="demo">A Paragraph.</p>

<button type="button" onclick="myFunction()">Try it</button>

</body>
</html>
```

Output:

Demo JavaScript in Head

A Paragraph.

Try it

Demo JavaScript in Head

Paragraph changed.

Try it

JavaScript Basics

JavaScript in <body>

- In this example, a JavaScript **function** is placed in the **<body>** section of an HTML page.
- The function is invoked (called) when a button is clicked:

JavaScript Basics

```
<!DOCTYPE html>
<html>
<body>

<h2>Demo JavaScript in Body</h2>

<p id="demo">A Paragraph.</p>

<button type="button" onclick="myFunction()">Try it</button>

<script>
function myFunction() {
  document.getElementById("demo").innerHTML = "Paragraph changed.";
}
</script>

</body>
</html>
```

Output:

Demo JavaScript in Body

A Paragraph.

Try it

Demo JavaScript in Body

Paragraph changed.

Try it

JavaScript Basics

External JavaScript:

- Scripts can also be placed in external files:

```
function myFunction() {  
    document.getElementById("demo").innerHTML = "Paragraph changed.";  
}
```

- External scripts are practical when the same code is used in many different web pages.
- JavaScript files have the file extension **.js**.
- To use an external script, put the name of the script file in the **src** (source) attribute of a **<script>** tag:

```
<script src="myScript.js"></script>
```

- An external script reference can be placed in **<head>** or **<body>** as user likes.
- The script will behave as if it was located exactly where the **<script>** tag is located.
- External scripts cannot contain **<script>** tags.

JavaScript Basics

External JavaScript Advantages

- ✓ Placing scripts in external files has some advantages:
 - ✓ It separates HTML and code
 - ✓ It makes HTML and JavaScript easier to read and maintain
 - ✓ Cached JavaScript files can speed up page loads
 - ✓ To add several script files to one page - use several script tags:
- An external script can be referenced in 3 different ways:
 - With a full URL (a full web address): `<script src="https://www.w3schools.com/js/myScript.js"></script>`
 - With a file path (like /js/): `<script src="/js/myScript.js"></script>`
 - Without any path: `<script src="myScript.js"></script>`

JavaScript Output

- JavaScript Display Possibilities
- JavaScript can "display" data in different ways:
 - Writing into an HTML element, using innerHTML.
 - Writing into the HTML output using document.write().
 - Writing into an alert box, using window.alert().
 - Writing into the browser console, using console.log().

Using innerHTML

Using innerHTML:

- To access an HTML element, JavaScript can use the `document.getElementById(id)` method.
- The `id` attribute defines the HTML element.
- The `innerHTML` property defines the HTML content.

Using innerHTML

EXAMPLE:

```
<!DOCTYPE html>
<html>
<body>

<h2>My First Web Page</h2>
<p>My First Paragraph.</p>

<p id="demo"></p>

<script>
document.getElementById("demo").innerHTML = 5 + 6;
</script>

</body>
</html>
```

Output:

My First Web Page

My First Paragraph.

Using document.write

EXAMPLE #1:

```
<!DOCTYPE html>
<html>
<body>

<h1>My First Web Page</h1>
<p>My first paragraph.</p>

<script>
document.write(5 + 6);
</script>

</body>
</html>
```

Output:

My First Web Page

My First Paragraph.

11

Using document.write

EXAMPLE #2:

```
<!DOCTYPE html>  
<html>  
<body>
```

```
<h1>My First Web Page</h1>  
<p>My first paragraph.</p>
```

```
<button type="button" onclick="document.write(5 + 6)">Try it</button>
```

```
</body>  
</html>
```

Output:

My First Web Page

My first paragraph.

Try it

Using window.alert

EXAMPLE #1:

```
<!DOCTYPE html>
<html>
<body>

<h1>My First Web Page</h1>
<p>My first paragraph.</p>

<script>
window.alert(5 + 6);
</script>

</body>
</html>
```

Using window.alert

EXAMPLE #2:

```
<!DOCTYPE html>
<html>
<body>

<h1>My First Web Page</h1>
<p>My first paragraph.</p>

<script>
alert(5 + 6);
</script>

</body>
</html>
```

Using console.log

EXAMPLE #1:

- `<!DOCTYPE html>`
`<html>`
`<body>`

`<script>`
`console.log(5 + 6);`
`</script>`

`</body>`
`</html>`

Javascript Print

- JavaScript does not have any print object or print methods.
- You cannot access output devices from JavaScript.
- The only exception is that you can call the `window.print()` method in the browser to print the content of the current window.

Javascript Print

- `<!DOCTYPE html>`
`<html>`
`<body>`

`<button onclick="window.print()">Print this page</button>`

`</body>`
`</html>`

Statement

- A **computer program** is a list of "instructions" to be "executed" by a computer.
- In a programming language, these programming instructions are called **statements**.
- A **JavaScript program** is a list of programming **statements**.
- JavaScript statements are composed of:
 - Values, Operators, Expressions, Keywords, and Comments.
- This statement tells the browser to write "Hello Dolly." inside an HTML element with id="demo":
 - Example: `document.getElementById("demo").innerHTML = "Hello Dolly.";`
- Most JavaScript programs contain many JavaScript statements.
- The statements are executed, one by one, in the same order as they are written.

Statement

Semicolons (;)

- Semicolons separate JavaScript statements.
- Add a semicolon at the end of each executable statement. When separated by semicolons, multiple statements on one line are allowed.

White Space

- JavaScript ignores multiple spaces. You can add white space to your script to make it more readable.

Line Length and Line Breaks

- For best readability, programmers often like to avoid code lines longer than 80 characters.
- If a JavaScript statement does not fit on one line, the best place to break it is after an operator

Code Blocks

- JavaScript statements can be grouped together in code blocks, inside curly brackets {...}.
- The purpose of code blocks is to define statements to be executed together.
- One place you will find statements grouped together in blocks, is in JavaScript functions

JavaScript Keywords

Keyword	Description
var	Declares a variable
let	Declares a block variable
const	Declares a block constant
if	Marks a block of statements to be executed on a condition
switch	Marks a block of statements to be executed in different cases
for	Marks a block of statements to be executed in a loop
function	Declares a function
return	Exits a function
try	Implements error handling to a block of statements

JavaScript Values

JavaScript Values:

The JavaScript syntax defines two types of values:

- Fixed values: Fixed values are called **Literals**.
- Variable values: Variable values are called **Variables**.

JavaScript Literals

The two most important syntax rules for fixed values are:

- Numbers** are written with or without decimals
- Strings** are text, written within double or single quotes

JavaScript Variables

- In a programming language, **variables** are used to **store** data values.
- JavaScript uses the keywords **var**, **let** and **const** to **declare** variables.
- An **equal sign** is used to **assign values** to variables.
- In this example, x is defined as a variable. Then, x is assigned (given) the value 6:

```
eg. let x;  
    x = 6;
```

JavaScript Variables

- Variables are Containers for Storing Data
- JavaScript Variables can be declared in 4 ways:
 - ✓ Automatically
 - ✓ Using `var`
 - ✓ Using `let`
 - ✓ Using `const`

JavaScript Variables

1. Automatically:

- In this first example, **x**, **y**, and **z** are undeclared variables.
- They are automatically declared when first used:

```
<!DOCTYPE html>
<html>
<body>
<h1>JavaScript Variables</h1>

<p>In this example, x, y, and z are undeclared.</p>
<p>They are automatically declared when first used.</p>

<p id="demo"></p>

<script>
x = 5;
y = 6;
z = x + y;
document.getElementById("demo").innerHTML =
"The value of z is: " + z;
</script>

</body>
</html>
```

Output:

JavaScript Variables

In this example, x, y, and z are undeclared.

They are automatically declared when first used.

The value of z is: 11

JavaScript Variables

2. Using Var:

- The **var** keyword was used in all JavaScript code from 1995 to 2015.
- The **let** and **const** keywords were added to JavaScript in 2015.
- The **var** keyword should only be used in code written for older browsers.

```
<!DOCTYPE html>
<html>
<body>
<h1>JavaScript Variables</h1>

<p>In this example, x, y, and z are variables.</p>

<p id="demo"></p>

<script>
var x = 5;
var y = 6;
var z = x + y;
document.getElementById("demo").innerHTML =
"The value of z is: " + z;
</script>

</body>
</html>
```

Output:

JavaScript Variables

In this example, x, y, and z are variables.

The value of z is: 11

JavaScript Variables

3. Using let:

```
<!DOCTYPE html>
<html>
<body>
<h1>JavaScript Variables</h1>

<p>In this example, x, y, and z are variables.</p>

<p id="demo"></p>

<script>
let x = 5;
let y = 6;
let z = x + y;
document.getElementById("demo").innerHTML =
"The value of z is: " + z;
</script>

</body>
</html>
```

Output:

JavaScript Variables

In this example, x, y, and z are variables.

The value of z is: 11

JavaScript Variables

4. Using const:

```
<!DOCTYPE html>
<html>
<body>
<h1>JavaScript Variables</h1>

<p>In this example, x, y, and z are variables.</p>

<p id="demo"></p>

<script>
const x = 5;
const y = 6;
const z = x + y;
document.getElementById("demo").innerHTML =
"The value of z is: " + z;
</script>

</body>
</html>
```

Output:

JavaScript Variables

In this example, x, y, and z are variables.

The value of z is: 11

JavaScript Variables

When to Use var, let, or const?

- Always declare variables
- Always use **const** if the value should not be changed
- Always use **const** if the type should not be changed (Arrays and Objects)
- Only use **let** if you can't use **const**
- Only use **var** if you MUST support old browsers.

JavaScript Identifiers

- All JavaScript **variables** must be **identified** with **unique names**. These unique names are called **identifiers**.
- Identifiers can be short names (like x and y) or more descriptive names (age, sum, totalVolume).
- The general rules for constructing names for variables (unique identifiers) are:
 - ✓ Names can contain letters, digits, underscores, and dollar signs.
 - ✓ Names must begin with a letter.
 - ✓ Names can also begin with \$ and _ (but we will not use it in this tutorial).
 - ✓ Names are case sensitive (y and Y are different variables).
 - ✓ Reserved words (like JavaScript keywords) cannot be used as names.
- JavaScript identifiers are case-sensitive.

JavaScript Functions

- Functions are **fundamental building blocks** in all programming.
- Functions enable **better code organization**, modularity, and efficiency.
- Functions are **reusable block of code** designed to perform a particular task.
- Functions **execute** when they are "called" or "invoked".

JavaScript Function Syntax

```
function name(p1, p2, p3) { // code }
```

- A function is defined with the **function** keyword, followed by the function **name**, followed by parentheses (), followed by brackets { }.
- The **name** follows the naming rules for variables (letters, digits, ...).
- Optional **parameters** are listed inside parentheses: **(p1, p2, p3)**
- The **code** to be executed is listed inside curly brackets: **{ code }**
- Functions can optionally **return** a value back to the "caller".

JavaScript Functions

Why Functions: Advantages

- With functions you can reuse code
- You can write code that can be used many times.
- You can use the same code with different arguments, to produce different results.

JavaScript Functions

Function Invocation ():

The code inside the function will execute when "something" **invokes** (calls) the function:

- When it is invoked (called) from JavaScript code
- When an event occurs (a user clicks a button)
- Automatically (self invoked)
- The () operator invokes (calls) the function

Example:

```
<!DOCTYPE html>
<html>
<body>
<h1>JavaScript Functions</h1>

<p>Invoke (call) a function that converts from Fahrenheit to Celsius:</p>

<p id="demo"></p>

<script>
function toCelsius(f) {
    return (5/9) * (f-32);
}

let value = toCelsius(77);
document.getElementById("demo").innerHTML = value;
</script>

</body>
</html>
```

Output:

JavaScript Functions

Invoke (call) a function that converts from Fahrenheit to Celsius:

25

Arrow Functions

```
<!DOCTYPE html>
<html>
<body>
<h1>JavaScript Functions</h1>
<h2>Function Expression</h2>

<p id="demo"></p>

<script>
// myFunction computes the product of a and b
let myFunction = function(a, b) {return a * b};

let result = myFunction(4, 3);
document.getElementById("demo").innerHTML = "The product is: " + result;
</script>

</body>
</html>
```

Output: JavaScript Functions

Function Expression

The product is: 12

```
<!DOCTYPE html>
<html>
<body>

<h1>JavaScript Functions</h1>
<h2>The Arrow Function</h2>

<p>This example shows the syntax of an Arrow Function, and how to use it.</p>

<p id="demo"></p>

<script>
// myFunction computes the product of a and b
myFunction = (a, b) => a * b;

let result = myFunction(4, 5);
document.getElementById("demo").innerHTML = "The product is: " + result
</script>

</body>
</html>
```

Output: JavaScript Functions

The Arrow Function

This example shows the syntax of an Arrow Function, and how to use it.

The product is: 20

JavaScript Functions

Local Variables:

- Variables declared **within** a JavaScript function, become **LOCAL** to the function.
- Local variables can only be accessed from within the function.
- Since local variables are only recognized inside their functions, variables with the same name can be used in different functions.
- Local variables are created when a function starts, and deleted when the function is completed.

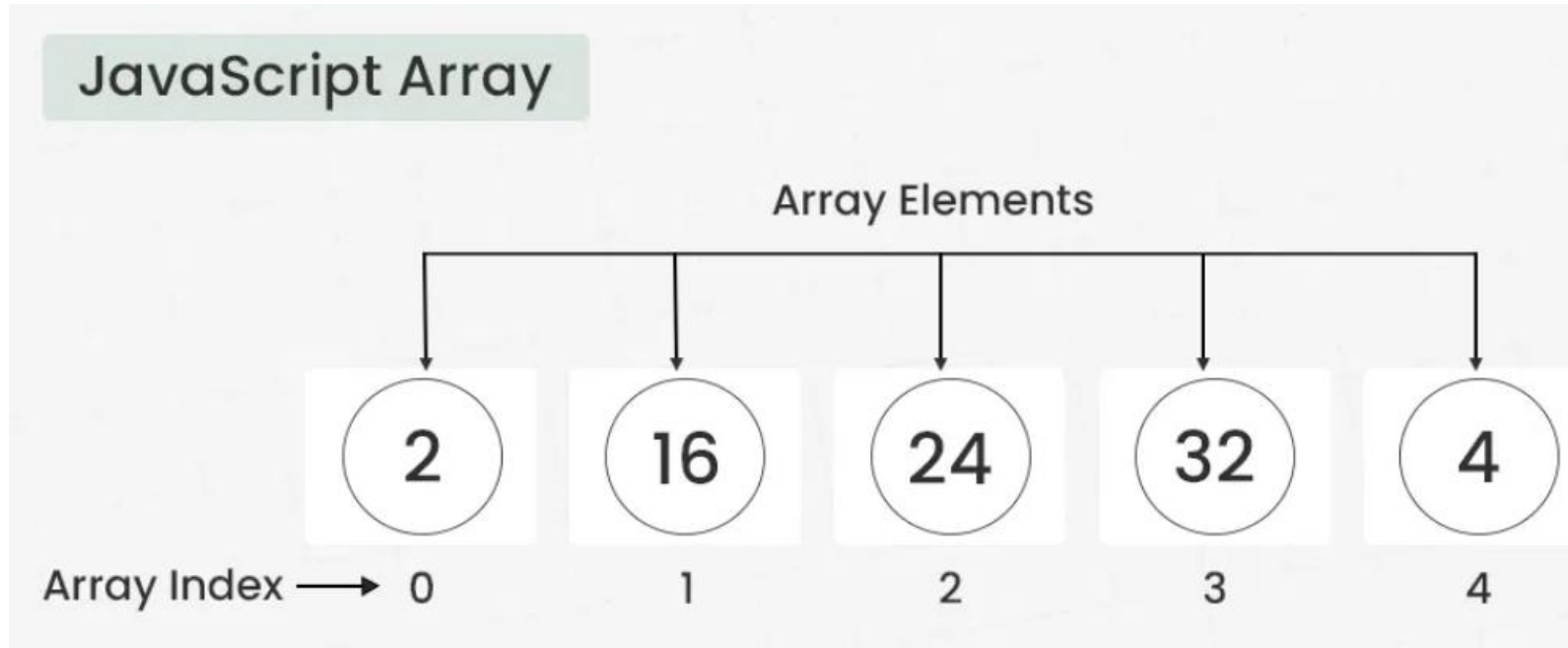
Parameters vs. Arguments:

- In JavaScript, function parameters and arguments are distinct concepts:
- **Parameters** are the **names** listed in the function definition.
- **Parameters** are the **names** of the values that will be passed.

```
function greet(name, age) {  
    return `Hello $name! You are ${age} years old.`;  
}
```

- "name" and "age" are **parameters**
- **Arguments** are the **values** passed to the function when it is invoked or called.
- **Arguments** are the **values** received by the function.
- Example: "John" and 21 are **arguments** `greet("John", 21);`

JavaScript Array



In JavaScript, an array is an ordered list of values. Each value is called an element specified by an index. An Array is an object type designed for storing data collections.

JavaScript Array

Key characteristics of JavaScript arrays are:

- **Elements:** An array is a list of values, known as elements.
- **Ordered:** Array elements are ordered based on their index.
- **Zero indexed:** The first element is at index 0, the second at index 1, and so on.
- **Dynamic size:** Arrays can grow or shrink as elements are added or removed.
- **Heterogeneous:** Arrays can store elements of different data types (numbers, strings, objects and other arrays).

JavaScript Array

Creating JavaScript arrays:

- The array literal form uses the square brackets [] to wrap a comma-separated list of elements.

```
let arrayName = [element1, element2, element3, ...];
```

- Array literal form uses the square brackets [] to wrap a comma-separated list of elements.
- The following example creates the colors array that holds string elements:

```
let colors = ['red', 'green', 'blue'];
```

- To create an empty array, you use square brackets without specifying any element like this:

```
let emptyArray = [];
```

JavaScript Array

Creating JavaScript arrays:

- Spaces and line breaks are not important. A declaration can span multiple lines.

```
const cars =  
  "Saab",  
  "Volvo",  
  "BMW"  
];
```

- You can also create an empty array, and provide elements later:

```
const cars = [];  
cars[0]= "Saab";  
cars[1]= "Volvo";  
cars[2]= "BMW";
```

- The JavaScript method **toString()** converts an array to a string of (comma separated) array values.

```
const fruits = ["Banana", "Orange", "Apple", "Mango"];  
document.getElementById("demo").innerHTML = fruits.toString();
```

- With JavaScript, the full array can be accessed by referring to the array name.

```
const cars = ["Saab", "Volvo", "BMW"];  
document.getElementById("demo").innerHTML = cars;
```

JavaScript Array

Accessing Array Elements

- JavaScript arrays are zero-based indexed. In other words, the first element of an array starts at index 0, the second element starts at index 1, and so on.
- To access an element in an array, you specify an index in the square brackets []
eg. arrayName[index]

```
<!DOCTYPE html>
<html>
<body>
<h1>JavaScript Arrays</h1>
<h2>Bracket Indexing</h2>

<p>JavaScript array elements are accessed using numeric indexes (starting from 0).</p>

<p id="demo"></p>

<script>
const cars = ["Saab", "Volvo", "BMW"];
document.getElementById("demo").innerHTML = cars[0];
</script>

</body>
</html>
```

Output:

JavaScript Arrays

Bracket Indexing

JavaScript array elements are accessed using numeric indexes (starting from 0).

Saab

JavaScript Array

Array Size:

- The **length** property of an array returns the length of an array (the number of array elements).
- The following example shows how to use the length property:

```
const fruits = ["Banana", "Orange", "Apple", "Mango"];  
let length = fruits.length;
```

Basic operations on arrays

- The following explains some basic operations on arrays. And you'll learn advanced operations such as `map()`, `filter()`, and `reduce()` in the next tutorials.
- 1) Adding an element to the end of an array
- To add an element to the end of an array, you use the `push()` method:

```
let seas = ['Black Sea', 'Caribbean Sea', 'North Sea', 'Baltic Sea'];  
seas.push('Red Sea');  
console.log(seas);
```

```
[ 'Black Sea', 'Caribbean Sea', 'North Sea', 'Baltic Sea', 'Red Sea' ]
```

JavaScript Array

2) **Adding an element to the beginning of an array**

- To add an element to the beginning of an array, you use the `unshift()` method:

```
let seas = ['Black Sea', 'Caribbean Sea', 'North Sea', 'Baltic Sea'];  
seas.unshift('Red Sea');  
console.log(seas);
```

```
[ 'Red Sea', 'Black Sea', 'Caribbean Sea', 'North Sea', 'Baltic Sea' ]
```

JavaScript Array

- Removing an element from the end of an array
- To remove an element from the end of an array, you use the `pop()` method:

```
let seas = ['Black Sea', 'Caribbean Sea', 'North Sea', 'Baltic Sea'];  
const lastElement = seas.pop();  
console.log(lastElement);
```

Baltic Sea

JavaScript Array

- Removing an element from the beginning of an array
- To remove an element from the beginning of an array, you use the `shift()` method:

```
let seas = ['Black Sea', 'Caribbean Sea', 'North Sea', 'Baltic Sea']; const  
firstElement = seas.shift();  
console.log(firstElement);
```

Black Sea

JavaScript Array

- Finding an index of an element in the array
- To find the index of an element, you use the `indexOf()` method:

```
let seas = ['Black Sea', 'Caribbean Sea', 'North Sea', 'Baltic Sea'];  
let index = seas.indexOf('North Sea');  
console.log(index); // 2
```

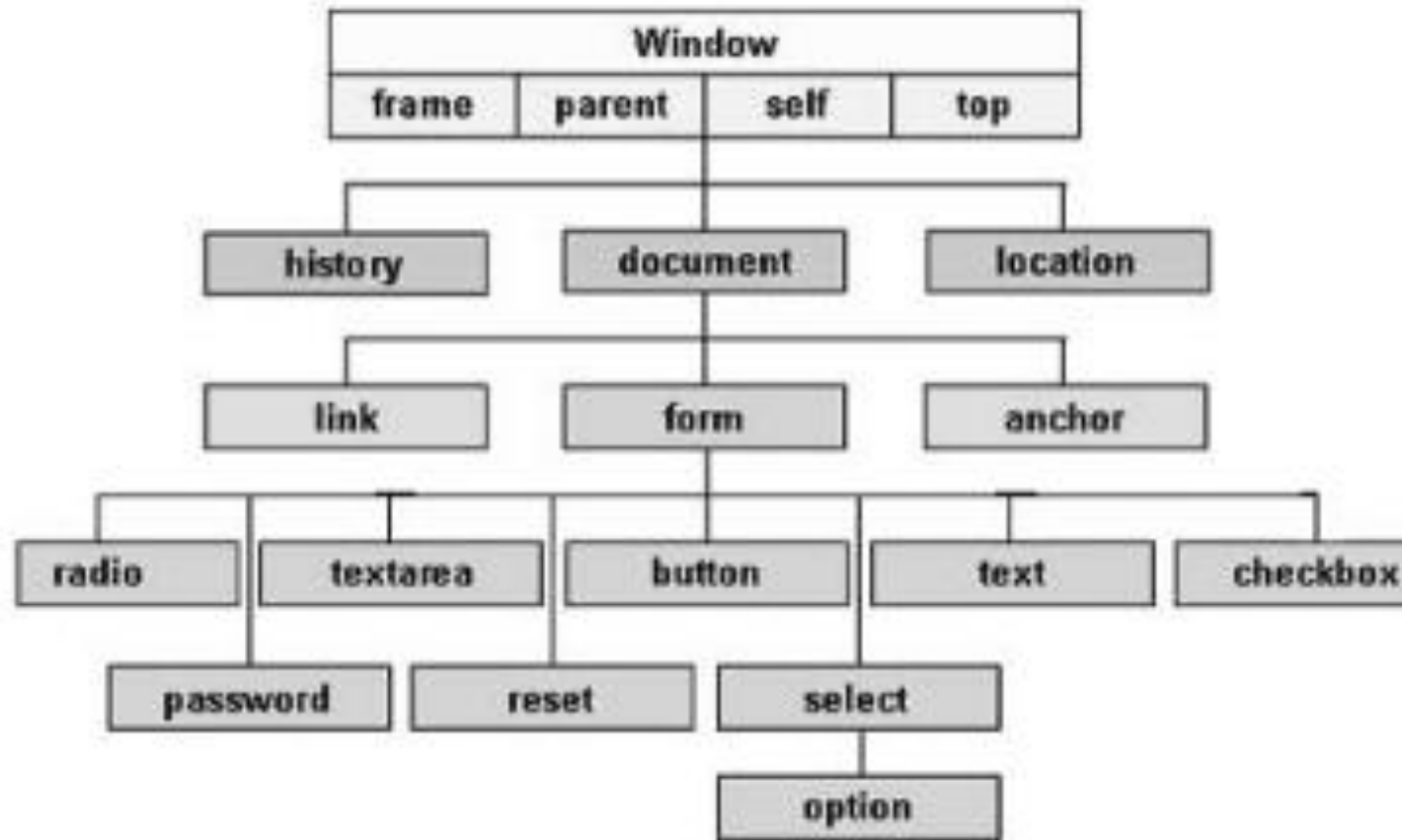
Summary

- In JavaScript, an array is an ordered list of values. Each value is called an element specified by an index.
- An array can hold values of mixed types.
- JavaScript arrays are dynamic, which means that they grow or shrink as needed.

DOM

- Every web page resides inside a browser window which can be considered as an object.
- A Document object represents the HTML document that is displayed in that window. The Document object has various properties that refer to other objects which allow access to and modification of document content.
- The way a document content is accessed and modified is called the **Document Object Model**, or **DOM**.

- The Objects are organized in a hierarchy. This hierarchical structure applies to the organization of objects in a Web document.
 - **Window object** – Top of the hierarchy. It is the outmost element of the object hierarchy.
 - **Document object** – Each HTML document that gets loaded into a window becomes a document object. The document contains the contents of the page.
 - **Form object** – Everything enclosed in the <form>...</form> tags sets the form object.
 - **Form control elements** – The form object contains all the elements defined for that object such as text fields, buttons, radio buttons, and checkboxes.



This hierarchical structure applies to the organization of objects in a Web document.

- There are several DOMs in existence. The following sections explain each of these DOMs in detail and describe how you can use them to access and modify document content.
 - The Legacy DOM – This is the model which was introduced in early versions of JavaScript language. It is well supported by all browsers, but allows access only to certain key portions of documents, such as forms, form elements, and images.
 - The W3C DOM – This document object model allows access and modification of all document content and is standardized by the World Wide Web Consortium (W3C). This model is supported by almost all the modern browsers.
 - The IE4 DOM – This document object model was introduced in Version 4 of Microsoft's Internet Explorer browser. IE 5 and later versions include support for most basic W3C DOM features.

The Legacy DOM

- This is the model which was introduced in early versions of JavaScript language. It is well supported by all browsers, but allows access only to certain key portions of documents, such as forms, form elements, and images.
- This model provides several read-only properties, such as title, URL, and lastModified provide information about the document as a whole. Apart from that, there are various methods provided by this model which can be used to set and get document property values.

Document Properties in Legacy DOM

Sr.No.	Property & Description
1	alinkColor Deprecated – A string that specifies the color of activated links. Ex – document.alinkColor
2	anchors[] An array of Anchor objects, one for each anchor that appears in the document Ex – document.anchors[0], document.anchors[1] and so on
3	applets[] An array of Applet objects, one for each applet that appears in the document Ex – document.applets[0], document.applets[1] and so on
4	bgColor Deprecated – A string that specifies the background color of the document. Ex – document.bgColor

5	cookie A string-valued property with special behavior that allows the cookies associated with this document to be queried and set. Ex – document.cookie
6	domain A string that specifies the Internet domain the document is from. Used for security purpose. Ex – document.domain
7	embeds[] An array of objects that represent data embedded in the document with the <embed> tag. A synonym for plugins[]. Some plugins and ActiveX controls can be controlled with JavaScript code. Ex – document.embeds[0], document.embeds[1] and so on
8	fgColor Deprecated - A string that specifies the default text color for the document Ex – document.fgColor
9	forms[] An array of Form objects, one for each HTML form that appears in the document. Ex – document.forms[0], document.forms[1] and so on

10	images[] An array of Image objects, one for each image that is embedded in the document with the HTML tag. Ex – document.images[0], document.images[1] and so on
11	lastModified A read-only string that specifies the date of the most recent change to the document Ex – document.lastModified
12	linkColor Deprecated – A string that specifies the color of unvisited links Ex – document.linkColor
13	links[] It is a document link array. Ex – document.links[0], document.links[1] and so on
14	location The URL of the document. Deprecated in favor of the URL property. Ex – document.location
15	plugins[] A synonym for the embeds[] Ex – document.plugins[0], document.plugins[1] and so on

16	Referrer A read-only string that contains the URL of the document, if any, from which the current document was linked. Ex – document.referrer
17	Title The text contents of the <title> tag. Ex – document.title
18	URL A read-only string that specifies the URL of the document. Ex – document.URL
19	vlinkColor Deprecated – A string that specifies the color of visited links. Ex – document.vlinkColor

Example

- We can locate any HTML element within any HTML document using HTML DOM. For instance, if a web document contains a **form** element then using JavaScript we can refer to it as **document.forms[0]**. If your Web document includes two **form** elements the first form is referred to as **document.forms[0]** and the second **document.forms[1]**.
- Using the hierarchy and properties given above, we can access the first form element using **document.forms[0].elements[0]** and so on.

- <html>
-
- <head>
- <title> Document Title </title>
-
- <script type = "text/javascript">
- <!--
- function myFunc() {
- var ret = document.title;
- alert("Document Title : " + ret);
-
- var ret = document.URL;
- alert("Document URL : " + ret);
-
- var ret = document.forms[0];
- alert("Document First Form : " + ret);
-
- var ret = document.forms[0].elements[1];
- alert("Second element : " + ret);
- }
- //-->
- </script>
-
- </head>

- <body>
- <h1 id = "title">This is main title</h1>
- <p>Click the following to see the result:</p>
-
- <form name = "FirstForm">
- <input type = "button" value = "Click Me" onclick = "myFunc();" />
- <input type = "button" value="Cancel">
- </form>
-
- <form name = "SecondForm">
- <input type = "button" value = "Don't ClickMe"/>
- </form>
-
- </body>
- </html>

The W3C DOM

- This document object model allows access and modification of all document content and is standardized by the World Wide Web Consortium (W3C). This model is supported by almost all the modern browsers.
- The W3C DOM standardizes most of the features of the legacy DOM and adds new ones as well. In addition to supporting forms[], images[], and other array properties of the Document object, it defines methods that allow scripts to access and manipulate any document element and not just special-purpose elements like forms and images.

Document Methods in W3C DOM

1	<code>createAttribute(name)</code> Returns a newly-created Attr node with the specified name. Ex – <code>document.createAttribute(name)</code>
2	<code>createComment(text)</code> Creates and returns a new Comment node containing the specified text. Ex – <code>document.createComment(text)</code>
3	<code>createDocumentFragment()</code> Creates and returns an empty DocumentFragment node. Ex – <code>document.createDocumentFragment()</code>
4	<code>createElement(tagName)</code> Creates and returns a new Element node with the specified tag name. Ex – <code>document.createElement(tagName)</code>

5	createTextNode(text) Creates and returns a new Text node that contains the specified text. Ex – document.createTextNode(text)
6	getElementById(id) Returns the Element of this document that has the specified value for its id attribute, or null if no such Element exists in the document. Ex – document.getElementById(id)
7	getElementsByName(name) Returns an array of nodes of all elements in the document that have a specified value for their name attribute. If no such elements are found, returns a zero-length array. Ex – document.getElementsByName(name)
8	getElementsByTagName(tagname) Returns an array of all Element nodes in this document that have the specified tag name. The Element nodes appear in the returned array in the same order they appear in the document source. Ex – document.getElementsByTagName(tagname)
9	importNode(importedNode, deep) Creates and returns a copy of a node from some other document that is suitable for insertion into this document. If the deep argument is true, it recursively copies the children of the node too. Supported in DOM Version 2 Ex – document.importNode(importedNode, deep)

Example

- This is very easy to manipulate (Accessing and Setting) document element using W3C DOM. You can use any of the methods like **getElementById**, **getElementsByName**, or **getElementsByTagName**.

HTML DOM Document getElementById()

- The `getElementById()` method returns an element with a specified value.
- The `getElementById()` method returns `null` if the element does not exist.
- The `getElementById()` method is one of the most common methods in the HTML DOM. It is used almost every time you want to read or edit an HTML element.
- **Example:**

```
<!DOCTYPE html>
<html>
<body>

<h1>The Document Object</h1>
<h2>The getElementById() Method</h2>

<p id="demo"></p>

<script>
document.getElementById("demo").innerHTML = "Hello World";
</script>

</body>
</html>
```

Output:

The Document Object

The getElementById() Method

Hello World

HTML DOM Document getElementByName()

- The `getElementByName()` method returns a collection of elements with a specified name.
- The `getElementByName()` method returns a live [NodeList](#).

NodeList:

- A [NodeList](#) is an array-like collection (list) of nodes.
- The nodes in the list can be accessed by index. The index starts at 0.
- [The length Property](#) returns the number of nodes in the list.

HTML DOM Document getElementByName()

Example:

```
<!DOCTYPE html>
<html>
<body>

<h1>The Document Object</h1>
<h2>The document.getElementsByName Method</h2>

Cats:
<input name="animal" type="checkbox" value="Cats">
Dogs:
<input name="animal" type="checkbox" value="Dogs">

<p>Number of elements with name="animal" is:</p>
<p id="demo"></p>

<script>
let num = document.getElementsByName("animal").length;
document.getElementById("demo").innerHTML = num;
</script>

</body>
</html>
```

Output:

The Document Object

The document.getElementsByName Method

Cats: ☐ Dogs: ☐

Number of elements with name="animal" is:

HTML DOM Document getElementByTagName()

- The `getElementsByTagName()` method returns a collection of all elements with a specified tag name.
- The `getElementsByTagName()` method returns an [HTMLCollection](#).
- The `getElementsByTagName()` property is read-only.
- `getElementsByTagName("*")` returns all elements in the document.

HTMLCollection

- An [HTMLCollection](#) is an array-like collection (list) of HTML elements.
- The [length Property](#) returns the number of elements in the collection.
- The elements can be accessed by index (starts at 0).
- An HTMLCollection is live. It is automatically updated when the document is changed.

HTML DOM Document getElementByName()

Example:

```
<!DOCTYPE html>
<html>
<body>

<h1>The Document Object</h1>
<h2>The getElementByTagName() Method</h2>

<p>This is a paragraph.</p>
<p>This is a paragraph.</p>
<p>This is a paragraph.</p>

<script>
document.getElementsByTagName("p")[0].innerHTML = "Hello World!";
</script>

</body>
</html>
```

Output:

The Document Object

The getElementByTagName() Method

Hello World!

This is a paragraph.

This is a paragraph.

- <html>
-
- <head>
- <title> Document Title </title>
-
- <script type = "text/javascript">
- <!--
- function myFunc() {
- var ret =
- document.getElementsByTagName("title");
- alert("Document Title : " + ret[0].text
-);
-
- var ret =
- document.getElementById("heading");
- alert(ret.innerHTML);
- }
- /-->
- </script>
-
- </head>

- <body>
- <h1 id = "heading">This is main title</h1>
- <p>Click the following to see the
- result:</p>
-
- <form id = "form1" name = "FirstForm">
- <input type = "button" value = "Click Me"
- onclick = "myFunc();" />
- <input type = "button" value = "Cancel">
- </form>
-
- <form d = "form2" name = "SecondForm">
- <input type = "button" value = "Don't
- ClickMe"/>
- </form>
-
- </body>
- </html>

The IE 4 DOM

- This document object model was introduced in Version 4 of Microsoft's Internet Explorer browser. IE 5 and later versions include support for most basic W3C DOM features.

Document Properties in IE 4 DOM

1	activeElement A read-only property that refers to the input element that is currently active (i.e., has the input focus). Ex – document.activeElement
2	all[] An array of all Element objects within the document. This array may be indexed numerically to access elements in source order, or it may be indexed by element id or name. Ex – document.all[]
3	charset The character set of the document. Ex – document.charset
4	children[] An array that contains the HTML elements that are the direct children of the document. Note that this is different from the all [] array that contains all the elements in the document, regardless of their position in the containment hierarchy. Ex – document.children[]

5	defaultCharset The default character set of the document. Ex – document.defaultCharset
6	expando This property, if set to false, prevents client-side objects from being expanded. Ex – document.expando
7	parentWindow The window that contains the document. Ex – document.parentWindow
8	readyState Specifies the loading status of a document. It has one of the following four string values – Ex – document.readyState
9	uninitialized The document has not started loading. Ex – document uninitialized

9	uninitialized The document has not started loading. Ex – document.uninitialized
10	loading The document is loading. Ex – document.loading
11	interactive The document has loaded sufficiently for the user to interact with it. Ex – document.interactive
12	complete The document is completely loaded. Ex – document.complete

Document Methods in IE4 DOM

Sr.No.	Property & Description
1	elementFromPoint(x,y) Returns the Element located at a specified point. Example: <code>document.elementFromPoint(x,y)</code>

Example

- The IE 4 DOM does not support the **getElementById()** method. Instead, it allows you to look up arbitrary document elements by id attribute within the all [] array of the document object.
- Here's how to find all tags within the first tag. Note that you must specify the desired HTML tag name in uppercase with the **all.tags()** method.

Example

- The IE 4 DOM does not support the **getElementById()** method. Instead, it allows you to look up arbitrary document elements by id attribute within the all [] array of the document object.
- Here's how to find all tags within the first tag. Note that you must specify the desired HTML tag name in uppercase with the **all.tags()** method.

- `var lists = document.all.tags("UL");`
- `var items = lists[0].all.tags("LI");`

Example

- `<html>`
-
- `<head>`
- `<title> Document Title </title>`
-
- `<script type = "text/javascript">`
- `<!--`
- `function myFunc() {`
- `var ret = document.all["heading"];`
- `alert("Document Heading : " + ret.innerHTML);`
-
- `var ret = document.all.tags("P");`
- `alert("First Paragraph : " + ret[0].innerHTML);`
- `}`
- `//-->`
- `</script>`
-
- `</head>`

- `<body>`
- `<h1 id = "heading">This is main title</h1>`
- `<p>Click the following to see the result:</p>`
-
- `<form id = "form1" name = "FirstForm">`
- `<input type = "button" value = "Click Me" onclick = "myFunc();" />`
- `<input type = "button" value = "Cancel">`
- `</form>`
-
- `<form d = "form2" name = "SecondForm">`
- `<input type = "button" value = "Don't ClickMe"/>`
- `</form>`
-
- `</body>`
- `</html>`

Built-in objects in JavaScript

- **Built-in objects are as follows:**
 1. Math Object
 2. Number Object
 3. Date Object
 4. Boolean Object
 5. String Objects
 6. Window Object
- These objects are used for simple data processing in the JavaScript.

Built-in objects in JavaScript

1.Math Object

- Math object is a built-in static object.
- It is used for performing complex math operations.
- For example...
`document.write(math.min(4.5,7,8));`
- The above statement will result in 4.5

Built-in objects in JavaScript

Math Methods

Methods	Description
<code>abs()</code>	Returns the absolute value of a number.
<code>acos()</code>	Returns the arccosine (in radians) of a number.
<code>ceil()</code>	Returns the smallest integer greater than or equal to a number.
<code>cos()</code>	Returns cosine of a number.
<code>floor()</code>	Returns the largest integer less than or equal to a number.
<code>log()</code>	Returns the natural logarithm (base E) of a number.
<code>max()</code>	Returns the largest of zero or more numbers.
<code>min()</code>	Returns the smallest of zero or more numbers.
<code>pow()</code>	Returns base to the exponent power, that is base exponent.

Built-in objects in JavaScript

Properties available on *Math* object are:

Property	Description
PI	The value of Pi
E	The base of natural logarithm e
LN2	Natural logarithm of 2
LN10	Natural logarithm of 10
LOG2E	Base 2 logarithm of e
LOG10E	Base 10 logarithm of e
SQRT2	Square root of 2
SQRT1_2	Square root of 1/2

Built-in objects in JavaScript

Method	Description
max(a,b)	Returns largest of a and b
min(a,b)	Returns least of a and b
round(a)	Returns nearest integer
ceil(a)	Rounds up. Returns the smallest integer greater than or equal to a
floor(a)	Rounds down. Returns the largest integer smaller than or equal to a
exp(a)	Returns e^a
pow(a,b)	Returns a^b
abs(a)	Returns absolute value of a
random()	Returns a pseudo random number between 0 and 1
sqrt(a)	Returns square root of a
sin(a)	Returns sin of a (a is in radians)
cos(a)	Returns cos of a (a is in radians)

Built-in objects in JavaScript

```
<html>
  <head>
    <title>JavaScript Math Object Methods</title>
  </head>
  <body>
<h3> Square root of 100 is </h3>
    <script type="text/javascript">

      var num=100;
      document.write("<h3> " +Math.sqrt(num)+"</h3>");
      document.write("<h3> " +Math.ceil(num)+"</h3>");
      document.write("<h3> " +Math.floor(num)+"</h3>");

    </script>
  </body>
</html>
```

Built-in objects in JavaScript

2. Number Object

- JavaScript Number is a built-in wrapper object which is used to work with numerical values.
- A Number object can be created using the Number() constructor.
- If we use the Number() constructor with the new keyword we can create the Number object.
- For example

```
let num = new Number(SOME_NUMERIC_VALUE);
```


Built-in objects in JavaScript

Number Object Methods

Sr.No.	Method & Description
1	<code>toExponential()</code> ↗ Forces a number to display in exponential notation, even if the number is in the range in which JavaScript normally uses standard notation.
2	<code>toFixed()</code> ↗ Formats a number with a specific number of digits to the right of the decimal.
3	<code>toLocaleString()</code> ↗ Returns a string value version of the current number in a format that may vary according to a browser's local settings.
4	<code>toPrecision()</code> ↗ Defines how many total digits (including digits to the left and right of the decimal) to display of a number.
5	<code>toString()</code> ↗ Returns the string representation of the number's value.
6	<code>valueOf()</code> ↗ Returns the number's value.

Built-in objects in JavaScript

Example

```
<html>
  <head>
    <title>Using Number Object</title>
  </head>
  <body>
    <script type="text/javascript">
      let num = new Number('18.907');
      document.write(num.toExponential() + "<br/>");
      document.write(num.toFixed() + "<br/>");
      document.write(num.toPrecision(3) + "<br/>");
      document.write(typeof num.toString() + "<br/>");
      document.write(num.valueOf() + "<br/>");
    </script>
  </body>
</html>
```

Output:

```
1.8907e+1
19
18.9
string
18.907
```

Built-in objects in JavaScript

3.Date Object

- Date is a data type.
- Date object manipulates date and time.
- Date() constructor takes no arguments.
- Date object allows you to get and set the year, month, day, hour, minute, second and millisecond fields.

Syntax:

```
var variable_name = new Date();
```

Example:

```
var current_date = new Date();
```

Built-in objects in JavaScript

Date Object Methods

Methods	Description
Date()	Returns current date and time.
getDate()	Returns the day of the month.
getDay()	Returns the day of the week.
getFullYear()	Returns the year.
getHours()	Returns the hour.
getMinutes()	Returns the minutes.
getSeconds()	Returns the seconds.
getMilliseconds()	Returns the milliseconds.
getTime()	Returns the number of milliseconds since January 1, 1970 at 12:00 AM.
getTimezoneOffset()	Returns the timezone offset in minutes for the current locale.
getMonth()	Returns the month.
setDate()	Sets the day of the month.
setFullYear()	Sets the full year.
setHours()	Sets the hours.
setMinutes()	Sets the minutes.
setSeconds()	Sets the seconds.
setMilliseconds()	Sets the milliseconds.

Built-in objects in JavaScript

Date Object Methods

<code>setTime()</code>	Sets the number of milliseconds since January 1, 1970 at 12:00 AM.
<code>setMonth()</code>	Sets the month.
<code>toString()</code>	Returns the date portion of the Date as a human-readable string.
<code>toLocaleString()</code>	Returns the Date object as a string.
<code>toGMTString()</code>	Returns the Date object as a string in GMT timezone.
<code>valueOf()</code>	Returns the primitive value of a Date object.

Built-in objects in JavaScript

```
<html>
  <body>
    <center>
      <h2>Date Methods</h2>
      <script type="text/javascript">
        var d = new Date();
        document.write("<b>Locale String:</b> " + d.toLocaleString()+"<br>");
        document.write("<b>Hours:</b> " + d.getHours()+"<br>");
        document.write("<b>FullYear:</b> " + d.getFullYear()+"<br>");
        document.write("<b>Minutes:</b> " + d.getMinutes()+"<br>");
      </script>
    </center>
  </body>
</html>
```

Output:

Date Methods

Locale String: 24/4/2023, 8:57:07 pm

Hours: 20

FullYear: 2023

Minutes: 57

Built-in objects in JavaScript

4.Boolean Object

- The **Boolean** object represents two values, either "true" or "false".
- If *value* parameter is omitted or is 0, -0, null, false, **NaN**, undefined, or the empty string (""), the object has an initial value of false.

Syntax

- Use the following syntax to create a **boolean** object.

```
var val = new Boolean(value);
```

Built-in objects in JavaScript

```
<html>
  <head>
    <title>JavaScript valueOf() Method</title>
  </head>
  <body>
    <script type = "text/javascript">
      var flag = new Boolean(false);
      document.write( "flag.valueOf is : " + flag.valueOf() );
    </script>
  </body>
</html>
```

Output

flag.valueOf is : false

Built-in objects in JavaScript

5.String Object

- String objects are used to work with text.
- It works with a series of characters.
- **Syntax:**
`var variable_name = new String(string);`

Example:

```
var s = new String(string);
```

Built-in objects in JavaScript

String Object Methods

Methods	Description
<code>charAt()</code>	It returns the character at the specified index.
<code>charCodeAt()</code>	It returns the ASCII code of the character at the specified position.
<code>concat()</code>	It combines the text of two strings and returns a new string.
<code>indexOf()</code>	It returns the index within the calling String object.
<code>match()</code>	It is used to match a regular expression against a string.
<code>replace()</code>	It is used to replace the matched substring with a new substring.
<code>search()</code>	It executes the search for a match between a regular expression.
<code>slice()</code>	It extracts a session of a string and returns a new string.
<code>split()</code>	It splits a string object into an array of strings by separating the string into the substrings.
<code>toLowerCase()</code>	It returns the calling string value converted lower case.
<code>toUpperCase()</code>	Returns the calling string value converted to uppercase.

Built-in objects in JavaScript

```
<html>
  <body>
    <center>
      <script type="text/javascript">
        var str = "CareerRide Info";
        var s = str.split();
        document.write("<b>Char At:</b> " + str.charAt(1)+"<br>");
        document.write("<b>CharCode At:</b> " + str.charCodeAt(2)+"<br>");
        document.write("<b>Index of:</b> " + str.indexOf("ide")+"<br>");
        document.write("<b>Lower Case:</b> " + str.toLowerCase()+"<br>");
        document.write("<b>Upper Case:</b> " + str.toUpperCase()+"<br>");
      </script>
    </center>
  </body>
</html>
```

Output:

Char At: a
CharCode At: 114
Index of: 7
Lower Case: careerride info
Upper Case: CAREERRIDE INFO

Built-in objects in JavaScript

6. Window Object

Window Object

- The **window object** represents a window in browser. An object of window is created automatically by the browser.
- Window is the object of browser, **it is not the object of javascript**. The javascript objects are string, array, date etc.

Built-in objects in JavaScript

Window Object Methods

Method	Description
alert()	displays the alert box containing message with ok button.
confirm()	displays the confirm dialog box containing message with ok and cancel button.
prompt()	displays a dialog box to get input from the user.
open()	opens the new window.
close()	closes the current window.
setTimeout()	performs action after specified time like calling function, evaluating expressions etc.

Built-in objects in JavaScript

7. Array Object

The properties available on *Array* object are:

Property	Description
length	Returns the number of elements in the array
constructor	Returns the function that created the array object
prototype	Allows us to add properties and methods to an array object

Built-in objects in JavaScript

Methods available on *Array* object are:

Method	Description
reverse()	Reverses the array elements
concat()	Joins two or more arrays
sort()	Sort the elements of an array
push()	Appends one or more elements at the end of an array
pop()	Removes and returns the last element
shift()	Removes and returns the first element
unshift()	Adds one or more elements at the beginning of an array
join()	Combines the elements of an array by a specified separator
indexOf()	Returns the first index of specified element
lastIndexOf()	Returns the last index of specified element
slice(startindex, endindex)	Returns selected elements

Regular Expression

RegExp Object

- A regular expression is a **pattern** of characters.
- The pattern is used to do pattern-matching "search-and-replace" functions on text.
- In JavaScript, a **RegExp Object** is a pattern with **Properties** and **Methods**.

Syntax

/pattern/modifier(s);

Regular Expression

- A **Regular Expression** is a sequence of characters that forms a **search pattern**.
- **Regex** is a common shorthand for a regular expression.
- JavaScript **RegExp** is an **Object** for handling Regular Expressions.
- RegExp are be used for:
 - ✓ Text searching
 - ✓ Text replacing
 - ✓ Text validation

Regular Expression

```
<!DOCTYPE html>
<html>
<body>
<h1>Regular Expressions</h1>
```

```
<p>Search a string for "vitbhopal", and display the position of
the match:</p>
```

```
<p id="demo"></p>
```

```
<script>
let text = "Visit vitbhopal!";
let n = text.search(/ vitbhopal /i);
document.getElementById("demo").innerHTML = n;
</script>
```

```
</body>
</html>
```

Output:

Regular Expressions

Search a string for "vitbhopal", and display the position of the match:

Regular Expression

- **Using String Methods:**

Regular expressions are often used with the **string methods**:

Method	Description
<code>match(regex)</code>	Returns an Array of results
<code>replace(regex)</code>	Returns a new String
<code>search(regex)</code>	Returns the index of the first match

JavaScript Errors

(Exceptions)

Throw, and Try...Catch...Finally

- The try statement defines a code block to run (to try).
- The catch statement defines a code block to handle any error.
- The finally statement defines a code block to run regardless of the result.
- The throw statement defines a custom error.

```
<!DOCTYPE html>
<html>
<body>
<h2>JavaScript Error Handling</h2>
<p>How to use <b>catch</b> to display an error.</p>
<p id="demo"></p>
<script>
try {
  addlert("Welcome guest!");
}
catch(err) {
  document.getElementById("demo").innerHTML = err.message;
}
</script>
</body>
</html>
```

Output:

JavaScript Error Handling

How to use **catch** to display an error.

addlert is not defined

JavaScript try and catch

- The try statement allows you to define a block of code to be tested for errors while it is being executed.
- The catch statement allows you to define a block of code to be executed, if an error occurs in the try block.
- The JavaScript statements try and catch come in pairs:

```
try {  
    Block of code to try  
}  
catch(err) {  
    Block of code to handle errors  
}
```

The throw Statement

- The throw statement allows you to create a custom error.
- Technically you can **throw an exception (throw an error)**.
- The exception can be a JavaScript String, a Number, a Boolean or an Object:

```
throw "Too big"; //throw a text
```

```
throw 500;      // throw a number
```

JavaScript Events

- The change in the state of an object is known as an **Event**.
- In html, there are various events which represents that some activity is performed by the user or by the browser.
- When [javascript](#) code is included in [HTML](#), js react over these events and allow the execution. This process of reacting over the events is called **Event Handling**. Thus, js handles the HTML events via **Event Handlers**.
- **For example**, when a user clicks over the browser, add js code, which will execute the task to be performed on the event.

Mouse events

Event Performed	Event Handler	Description
click	onclick	When mouse click on an element
mouseover	onmouseover	When the cursor of the mouse comes over the element
mouseout	onmouseout	When the cursor of the mouse leaves an element
mousedown	onmousedown	When the mouse button is pressed over the element
mouseup	onmouseup	When the mouse button is released over the element
mousemove	onmousemove	When the mouse movement takes place.

Keyboard events

Event Performed	Event Handler	Description
Keydown & Keyup	onkeydown & onkeyup	When the user press and then release the key

Form events

Event Performed	Event Handler	Description
focus	onfocus	When the user focuses on an element
submit	onsubmit	When the user submits the form
blur	onblur	When the focus is away from a form element
change	onchange	When the user modifies or changes the value of a form element

Window/Document events

Event Performed	Event Handler	Description
load	onload	When the browser finishes the loading of the page
unload	onunload	When the visitor leaves the current webpage, the browser unloads it
resize	onresize	When the visitor resizes the window of the browser

```
<html>
<head>
<h1> Javascript Events </h1>
</head>
<body>
<script language="Javascript" type="text/Javascript">
    function mouseoverevent()
    {
        alert("This is JavaTpoint");
    }
</script>
<p onmouseover="mouseoverevent()">
</body>
</html>
```

Javascript Events

Keep cursor over me



JavaScript Form Validation

- HTML form validation can be done by JavaScript.
- If a form field (fname) is empty, this function alerts a message, and returns false, to prevent the form from being submitted:

- JavaScript Example:

```
function validateForm() {  
    let x = document.forms["myForm"]["fname"].value;  
    if (x == "") {  
        alert("Name must be filled out");  
        return false;  
    }  
}
```

- The function can be called when the form is submitted:

```
<form name="myForm" action="/action_page.php" onsubmit="return validateForm()" method="post">  
Name: <input type="text" name="fname">  
<input type="submit" value="Submit">  
</form>
```

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Validation</h2>

<p>Please input a number between 1 and 10:</p>

<input id="numb">

<button type="button" onclick="myFunction()">Submit</button>

<p id="demo"></p>

<script>
function myFunction() {
  // Get the value of the input field with id="numb"
  let x = document.getElementById("numb").value;
  // If x is Not a Number or less than one or greater than 10
  let text;
  if (isNaN(x) || x < 1 || x > 10) {
    text = "Input not valid";
  } else {
    text = "Input OK";
  }
  document.getElementById("demo").innerHTML = text;
}
</script>

</body>
</html>
```

Output:

JavaScript Validation

Please input a number between 1 and 10: